

CS 650 – Full Stack Application Develop

6-1 Project One: Full Stack Architecture and Implementation

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

August 12, 2026

Part One: Design

Architectural Components

Northwind Works uses a layered architecture that separates presentation, communication, business logic, and persistence. The React SPA renders login, dashboard, client, consultant, and work-request views, including forms, filters, assignment controls, and protected navigation. React Router manages client-side routes, while a centralized API service performs authenticated HTTP requests, so UI components remain focused on state and interaction.

The Node.js/Express back end exposes the REST API, manages sessions, applies authentication middleware, and coordinates validation and responses. Routes delegate business decisions to a service layer. Major endpoints cover authentication, clients, consultants, work requests, status changes, and assignments. SQLite provides persistent relational storage.

Data Models and Storage

The relational database contains users, clients, consultants, work_requests, and assignments. Users store a unique username and bcrypt hash. Work requests reference clients and store title, description, priority, status, and timestamps. Assignments connect consultants to requests and record assignment dates. Foreign keys enforce relationships, and the assignment design supports one current consultant per request.

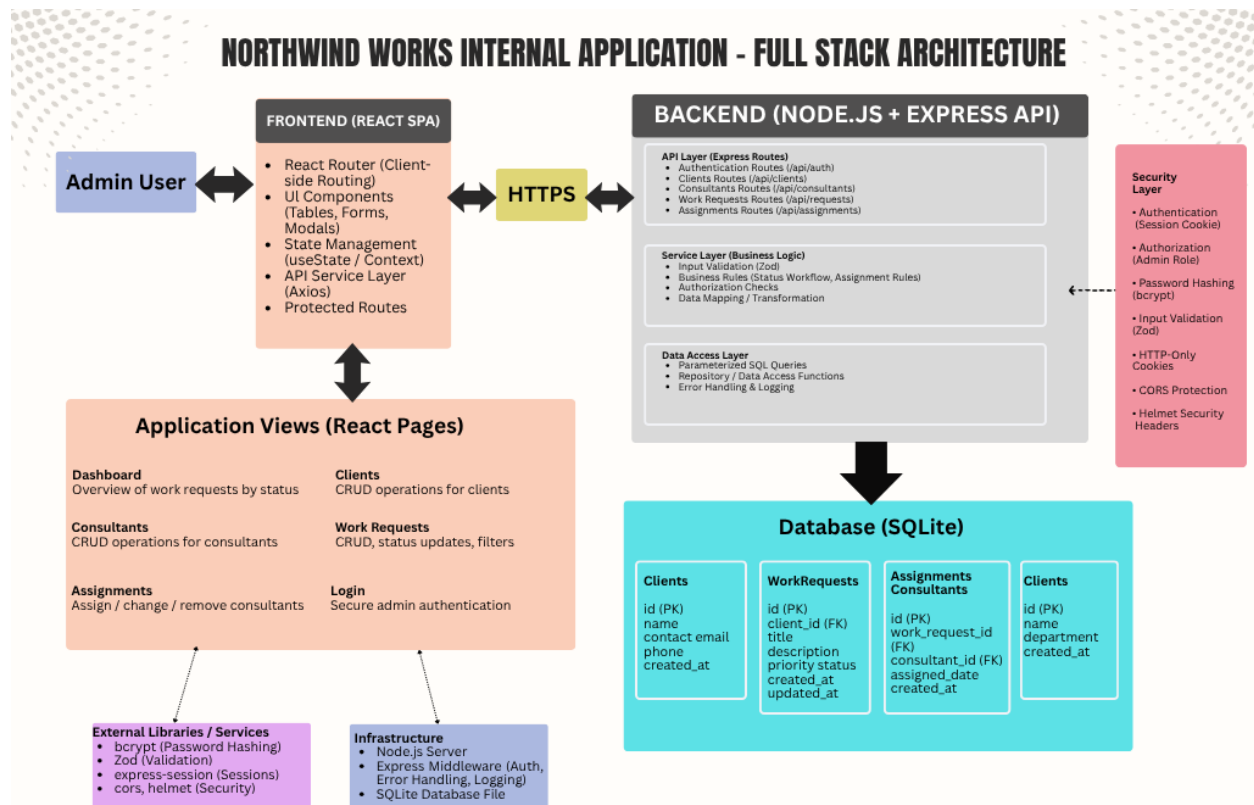
Core Workflow Requirements

The core workflow manages a client request from creation through assignment and completion or cancellation. An authenticated admin can create, view, edit, filter, and delete requests; assign,

reassign, or remove consultants; and update status. New requests begin Open. Valid transitions are Open to In Progress or Cancelled and In Progress to Completed or Cancelled. Completed and Cancelled are final.

Secondary requirements support usability and integrity: the interface displays client, priority, description, status, and assignment information; selection controls use stored client and consultant data; protected resources require authentication; and input is validated. The browser exposes only valid status actions, while the service layer independently enforces the same rules, so direct API calls cannot bypass the workflow.

Full Stack Architecture Diagram



Industry Best Practices

Separation of concerns improves maintainability: React handles presentation, the API service handles HTTP communication, Express routes handle transport concerns, the service layer owns business rules, and SQLite owns persistence. Security is layered through bcrypt hashing, HTTP-only/SameSite cookies, environment-based secrets, Zod validation, parameterized data operations, authentication middleware, and foreign keys.

Part Two: Build

Framework Support for Full Stack Development

The stack directly supports the requirements. React provides reusable components, state-driven updates, forms, and filters; React Router supplies SPA navigation and protected routes; Vite provides development and build tooling; and the centralized API service standardizes data fetching and credential handling.

Express routing and middleware map naturally to REST resources. Authentication can be applied consistently while route handlers delegate database and workflow operations to the service layer. SQLite supplies relational tables, constraints, transactions, and persistence without a separate server. Together, the frameworks support the complete path from React through the REST API and service layer to SQLite and back.

Security Parameters

Exploit prevention treats client input as untrusted. Zod validates expected fields and values, parameterized SQL reduces injection risk, foreign keys protect relationships, and server-side status validation blocks prohibited transitions even when the UI is bypassed.

Authentication compares submitted credentials with a bcrypt hash and establishes a server-side session. Authorization follows the single Admin role: authenticated admins can use management functions, while unauthenticated users are blocked from protected API endpoints and SPA routes. Production hardening should add HTTPS, persistent sessions, CSRF protection, login rate limiting, centralized secrets, audit logging, and stronger credential policy.

Alternative Frameworks and Tradeoffs

Angular could replace React and provide stronger built-in conventions and dependency injection, but with greater complexity. Vue offers a simpler component model, but migration would add effort without closing a requirement gap. NestJS could add modules, controllers, providers, and dependency injection to the back end, but that abstraction is more valuable at larger scale.

PostgreSQL would be preferable for a highly concurrent production deployment because of centralized hosting, access control, backups, and stronger multi-user concurrency. SQLite remains appropriate here because it is simple to deploy while still demonstrating relational modeling, foreign keys, persistence, and integrity.

Implemented Full Stack Solution and API Strategy

The completed build implements work-request functionality across both stacks. The REST API supports listing, creating, updating, deleting, filtering, status changes, and assignments. The React SPA provides protected navigation, forms, filters, detail views, assignment controls, and status actions, while the API service centralizes communication with Express.

The work-request detail screen combines multiple models and rules. It displays client, priority, description, status, and current consultant. Open requests offer In Progress or Cancelled; In Progress requests offer Completed or Cancelled; final requests cannot move backward. The service layer validates transitions independently, so correctness does not depend on the UI.

Part Three: Validation

Validation Approach and Results

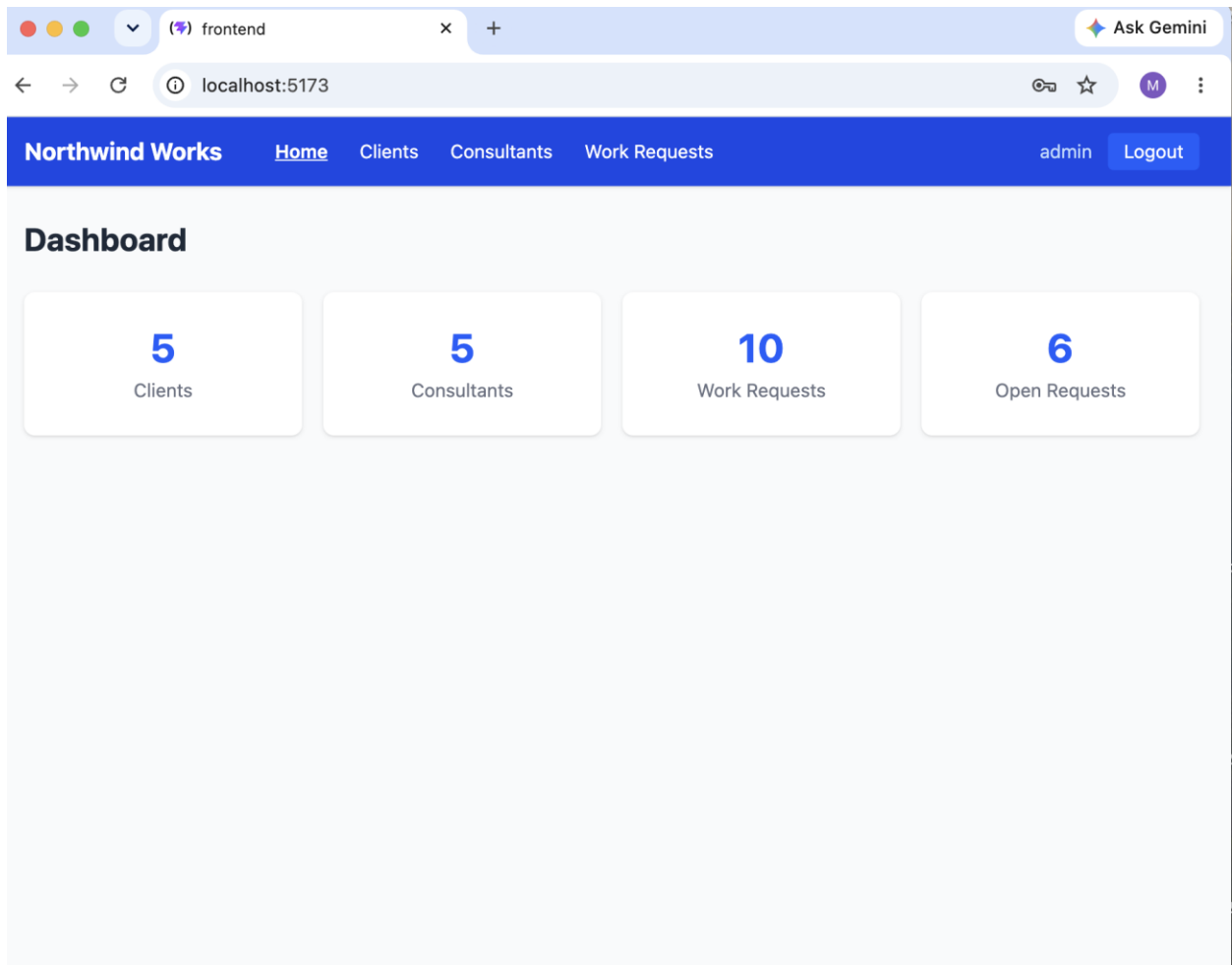
The final product was deployed locally and hand-tested against the acceptance criteria. SQLite was seeded with representative data, Express ran on localhost:3000, and React/Vite ran on localhost:5173. Realistic admin workflows exercised in the browser, API service, REST API, sessions, business rules, and database together.

Authentication passed when the admin reached the dashboard. Client creation passed when Test Client appeared in the client table. Work-request creation passed when Client Website Assessment was created for Test Client with High priority and Open status. Assignment passed when Alice Chen from Engineering persisted as the current consultant.

The status workflow passed from Open to In Progress to Completed. In Progress exposed only Completed or Cancelled, with no backward Open action. After completion, the detail view stated that the request was final and could not move backward, directly matching the required workflow.

Filtering passed independently and in combination: Completed reduced the list to completed requests and Completed plus Test Client returned only Client Website Assessment. After logging out, navigating the protected work-request route redirected to /login, confirming unauthenticated access is blocked.

Hand-Testing Evidence and Acceptance Mapping



Clients

[Add Client](#)

NAME	EMAIL	PHONE	
Acme Corp	contact@acme.com	555-1000	Edit Delete
Globex	info@globex.com	555-2000	Edit Delete
Initech	hello@initech.com	—	Edit Delete
Test Client	testclient@example.com	555-9999	Edit Delete
Umbrella LLC	ops@umbrella.io	555-3000	Edit Delete
Wayne Industries	procurement@wayne.com	555-4000	Edit Delete

← Work Requests

Client Website Assessment

Open

Client: Test Client

Priority: High

Review the client's current website and provide recommendations for improvements.

VALID STATUS ACTIONS

Move to In Progress

Move to Cancelled

Assignment

CURRENT CONSULTANT

No consultant assigned

Select consultant



Assign

Work Requests

Track client engagements, priorities, assignments, and progress.

Add Work Request

All statuses ▾

All clients ▾

TITLE	PRIORITY	STATUS	
Client Website Assessment	High	Open	Edit Delete
Website Redesign	High	In Progress	Edit Delete
API Integration	Medium	Open	Edit Delete
Security Audit	High	Completed	Edit Delete
Data Migration	Low	Cancelled	Edit Delete
Dashboard Build	Medium	Open	Edit Delete
Mobile App Prototype	High	Open	Edit Delete
Brand Guidelines	Low	In Progress	Edit Delete
Cost Analysis Report	Medium	Open	Edit Delete
SEO Overhaul	Medium	Open	Edit Delete
Staff Training Portal	Low	Open	Edit Delete

[← Work Requests](#)

Client Website Assessment

Completed

Client: Test Client

Priority: High

Review the client's current website and provide recommendations for improvements.

This request is in a final status and cannot move backward.

Assignment

CURRENT CONSULTANT

Alice Chen

Engineering

[Remove](#)[Reassign](#)

Work Requests

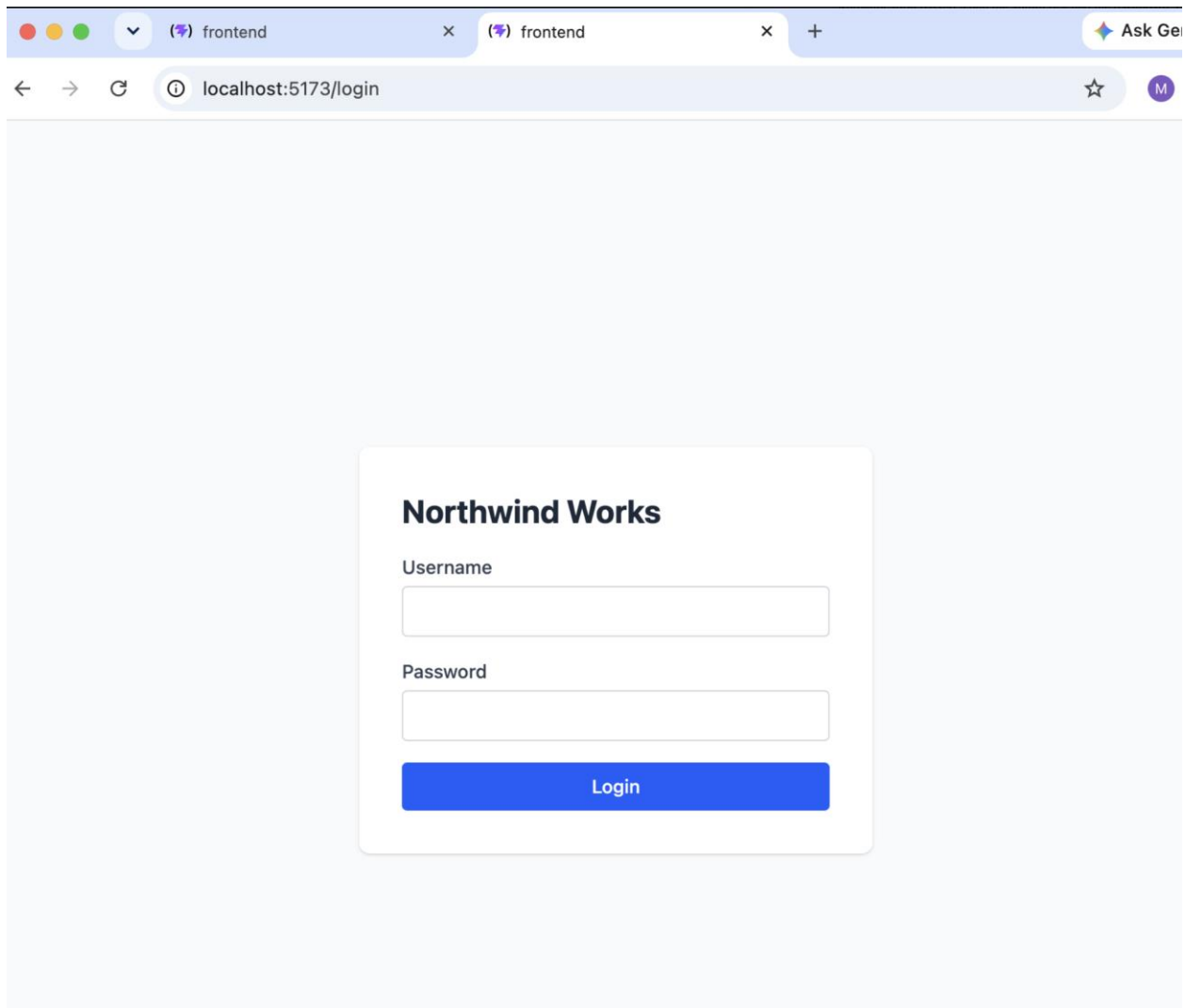
Track client engagements, priorities, assignments, and progress.

Add Work Request

Completed ▾

Test Client ▾

TITLE	PRIORITY	STATUS	
Client Website Assessment	High	Completed	Edit Delete



The evidence maps directly to acceptance criteria: authenticated dashboard access proves login; the completed request proves client association, consultant assignment, valid status progression, and final-state locking; combined filtering proves status/client filtering; and the login redirect proves protected routing.

Final Evaluation

The completed application satisfies the central functional and architectural expectations. It is full stack rather than interface-only: React communicates through a REST API, Express applies authentication and routing, the service layer enforces business rules, and SQLite persists in

relational data. Security is integrated through hashed passwords, sessions, validation, parameterized operations, and protected endpoints.

The project demonstrates the requirements of translation, architecture design, secure implementation, separation of concerns, relational modeling, API integration, and evidence-based validation. Future production work should add persistent sessions, HTTPS, CSRF defenses, rate limiting, audit logging, automated integration/regression tests, and a production database if concurrency grows.